RAG Chatbot with Campus Navigation

MERN + LangChain (JS) + OpenAI Embeddings + Google Maps

Lahore Garrison University (LGU) smart campus assistant

WHAT IS INSIDE

1 Project scope
2 What is RAG? (easy words)
3 Platforms overview
4 OpenAI setup: account, credits, API key
5 MongoDB Atlas setup
6 Project structure and install
7 Backend: ingest, RAG, location router
8 Frontend: chat UI and map card
9 Run and test
10 Presenting and defending your FYP
11 Troubleshooting and final checklist

Written for junior developers | October 2026

1 Project Scope

We are building a **smart campus chatbot** for Lahore Garrison University. A student types a question. The bot either answers from university knowledge (using RAG) or, if the question is about *where* something is, shows a map card with a red marker and an **Open in Google Maps** button.

In scope

Feature	Description
Knowledge chatbot (RAG)	Answers questions about LGU (admissions, departments, library, rules) using your own documents stored in a vector database.
Location router	If the message contains a known place (LGU, Cafe, Fountain Ground...) the frontend shows a map card instead of plain text.
Map card	Embedded Google Map with a red pin, plus buttons: Open in Google Maps and Get Directions.
MERN app	MongoDB Atlas, Express, React, Node.js. LangChain (JavaScript) handles the RAG logic.

Out of scope (mention as future work)

- Indoor navigation (room by room inside buildings).
- Live GPS tracking of buses or students.
- Voice assistant and Urdu language (good extensions, not needed for the first demo).

> **Known limitation, and how we remove it**
> You said you can only demo places that are already registered on Google Maps. We avoid this problem: our map card uses **latitude and longitude**, not the place registration. Any point on earth can get a red pin. You just right-click the spot on Google Maps and copy the coordinates (section 8.3).

2 What is RAG? (Easy Words)

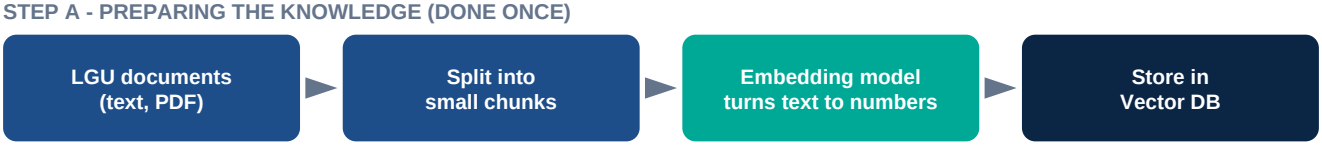
RAG = Retrieval-Augmented Generation. It is a way to make an AI chatbot answer using **your own data** instead of guessing.

The problem

ChatGPT-style models (LLMs) were trained on public internet data. They know nothing about LGU's private details, such as the admission schedule or the cafe timing. If you ask, they may **make up** a confident but wrong answer (called a hallucination).

The solution: an open-book exam

Imagine two students. Student A takes the exam from memory. Student B is allowed to open the book, **find the right page first**, then write the answer. RAG makes the chatbot Student B.



STEP B - EVERY TIME A USER ASKS

The words you must know

Term	Simple meaning
LLM	The chat brain (for example GPT). It writes the final answer.
Embedding	A list of numbers (a vector) that captures the *meaning* of a sentence. Similar meaning gives similar numbers.
Embedding model	The tool that creates embeddings. We use OpenAI **text-embedding-3-small**.
Vector database	A database that can search by meaning. We use MongoDB Atlas Vector Search, so it fits the MERN stack.
Chunk	A small piece of a document (about 500 characters) so search is precise.
Retriever	The part that finds the most relevant chunks for a question.
Prompt	The instruction we send to the LLM: 'Answer only from this context: ...'
LangChain	A JavaScript library that connects all the above pieces with ready-made helpers.

Why not just search keywords?
Keyword search fails if the user types different words. 'Where can I eat?' has no word in common with 'LGU Cafe', but their *meaning* is close, so vector search finds it. That is the power of embeddings.

3 Platforms Overview

Before coding, know every platform you will touch. Create these accounts first.

Platform	Role in the project	Cost
OpenAI Platform (platform.openai.com)	Gives the API key. Provides the embedding model and the chat model.	Pay-as-you-go prepaid credits. A few dollars is enough for an FYP.
MongoDB Atlas	Cloud database. Stores chunks and their embeddings. Runs vector search.	Free M0 cluster is enough.
Node.js + Express	Backend server. Exposes the POST /api/chat endpoint.	Free
LangChain JS	Library for splitting text, calling embeddings, retrieving and prompting.	Free (open source)
React (Vite)	Frontend chat window and the map card.	Free
Google Maps	Map display and navigation through simple URLs (no API key needed for our method).	Free
VS Code and Postman	Code editor and API testing.	Free

BIG PICTURE

- **Path 1 (fast, no AI):** message contains a known place, so the router returns coordinates. React draws the map card.
- **Path 2 (RAG):** everything else goes to LangChain. Embed the question, search Atlas, send the chunks to the LLM, return text.

4 OpenAI Setup: Account, Credits and API Key

Important
A ChatGPT Plus subscription is **not** the same as API access. The API is billed separately with prepaid credits on the Platform site.

4.1 Create the account

- Open **platform.openai.com** and click Sign up (Google or email).
- Verify your email. Phone verification may be requested.
- You land on the Dashboard. Create or select a **Project** (for example: lgu-chatbot). Keys live inside projects.

4.2 Add billing and fill credits

- Go to **Settings, then Billing**.
- Add a payment method (international debit or credit card).
- Click **Add to credit balance** and buy credits. The minimum is usually 5 USD. Credits are prepaid, so you can never be charged more than you load.
- Keep **auto-recharge OFF** while learning.
- Under **Limits**, set a small monthly budget (for example 5 USD) as a safety net.

Card problems in Pakistan?
Some local cards get declined. Try another international card or a virtual card from a local fintech app. If payment is impossible, ask a friend to fund the account, or switch to a free local embedding model through Ollama or Hugging Face (LangChain supports both; the code structure stays the same).

4.3 Create the API key

- Open **Dashboard, then API keys** and click **Create new secret key**.
- Name it (lgu-backend), choose your project, create it.
- **Copy it immediately.** It starts with sk- and is shown only once. If lost, create a new one.
- Paste it into the backend **.env** file (section 6). Never put it in React code, never push it to GitHub.

4.4 How much will it cost?

Embeddings are extremely cheap. Roughly: text-embedding-3-small costs about 0.02 USD per **one million** tokens, so embedding 100 pages costs a fraction of a cent. The chat model (we use gpt-4o-mini, a low-cost model) costs a small fraction of a cent per question. Check the live numbers on openai.com/api/pricing because prices change. A 5 USD top-up will cover your whole FYP.

5 MongoDB Atlas Setup

- Sign up at **mongodb.com/atlas** and create a free **M0 cluster**.
- **Database Access:** create a user (username and password).
- **Network Access:** add IP 0.0.0.0/0 for development only.
- Click **Connect, then Drivers**, and copy the connection string. Replace the password inside it.
- Create database **lgu** and collection **lgu_docs**. (Browse Collections, Create Database.)
- Open **Atlas Search, then Create Search Index, then Vector Search (JSON editor)**. Pick the lgu_docs collection, name the index **vector_index** and paste:

Atlas vector index definition

```
{
  "fields": [
    { "type": "vector", "path": "embedding", "numDimensions": 1536, "similarity": "cosine" }
  ]
}
```

Why 1536?
text-embedding-3-small produces vectors with 1536 numbers. The index size must match the model. If you change models later, rebuild the index and re-run ingestion.

6 Project Structure and Installation

Folder layout

```
lgu-chatbot/
  server/
    .env
    package.json
    server.js            <- Express app and /api/chat route
    rag.js               <- LangChain retriever + prompt + LLM
    locations.js         <- place list + keyword matcher (the router)
    ingest.js            <- run once to fill the vector DB
    data/knowledge.json  <- your LGU knowledge text
  client/                <- React (Vite)
    src/App.jsx
    src/components/ChatBox.jsx
    src/components/LocationCard.jsx
```

6.1 Backend install

Terminal

```
mkdir lgu-chatbot && cd lgu-chatbot
mkdir server && cd server
npm init -y
npm i express cors dotenv mongodb langchain @langchain/core @langchain/openai \
      @langchain/mongodb @langchain/textsplitters
```

In package.json add **"type": "module"** so we can use import syntax, and a script **"start": "node server.js"**.

server/.env

```
OPENAI_API_KEY=sk-paste-your-key-here
MONGODB_URI=mongodb+srv://USER:PASSWORD@cluster0.xxxxx.mongodb.net/
PORT=5000
```

Security rule
Create a **.gitignore** file containing **.env** and **node_modules**. If your key leaks on GitHub, delete it on the OpenAI dashboard at once and create a new one.

7 Backend: Step by Step

7.1 Step 1 - Write the knowledge file

Put real LGU information here: admissions, departments, fee rules, library timing, societies. Each item has **text** and optional **metadata**. The sample below is placeholder text. Replace it with real facts.

server/data/knowledge.json

```
[
  { "text": "LGU Cafe is the main food place on campus. Students can buy snacks, lunch and tea here.",
    "metadata": { "locationId": "cafe", "topic": "facilities" } },
  { "text": "The Fountain Ground is the open green area used for events and student gatherings.",
    "metadata": { "locationId": "fountain", "topic": "facilities" } },
  { "text": "The central library is open on weekdays. Students need their university card to enter.",
    "metadata": { "topic": "library" } }
]
```

7.2 Step 2 - Ingest (embed and store)

This script runs **once** (and again whenever you change the knowledge file). It splits text into chunks, asks OpenAI for an embedding of each chunk, and saves everything into Atlas.

server/ingest.js

```
import "dotenv/config";
import fs from "fs";
import { MongoClient } from "mongodb";
import { OpenAIEmbeddings } from "@langchain/openai";
import { MongoDBAtlasVectorSearch } from "@langchain/mongodb";
import { RecursiveCharacterTextSplitter } from "@langchain/textsplitters";

const client = new MongoClient(process.env.MONGODB_URI);
const collection = client.db("lgu").collection("lgu_docs");
const embeddings = new OpenAIEmbeddings({ model: "text-embedding-3-small" });

const raw = JSON.parse(fs.readFileSync("./data/knowledge.json", "utf-8"));
const splitter = new RecursiveCharacterTextSplitter({ chunkSize: 500, chunkOverlap: 50 });
const docs = await splitter.createDocuments(
  raw.map((r) => r.text),
  raw.map((r) => r.metadata)
);

await collection.deleteMany({}); // start clean
await MongoDBAtlasVectorSearch.fromDocuments(docs, embeddings, {
  collection, indexName: "vector_index", textKey: "text", embeddingKey: "embedding",
});
console.log(`Stored ${docs.length} chunks in Atlas`);
await client.close();
```

Run it

```
node ingest.js
```

Check Atlas, Browse Collections: you should see documents with a long **embedding** array. Wait about a minute for the vector index to become Active.

7.3 Step 3 - The RAG engine

Flow inside **askRag()**: question, embed, find 3 similar chunks, build a prompt, ask the LLM, return the answer.

server/rag.js

```
import "dotenv/config";
import { MongoClient } from "mongodb";
import { ChatOpenAI, OpenAIEmbeddings } from "@langchain/openai";
import { MongoDBAtlasVectorSearch } from "@langchain/mongodb";
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { StringOutputParser } from "@langchain/core/output_parsers";

const client = new MongoClient(process.env.MONGODB_URI);
const collection = client.db("lgu").collection("lgu_docs");
```

```
const vectorStore = new MongoDBAtlasVectorSearch(
  new OpenAIEmbeddings({ model: "text-embedding-3-small" }),
  { collection, indexName: "vector_index", textKey: "text", embeddingKey: "embedding" }
);
const retriever = vectorStore.asRetriever({ k: 3 });

const llm = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0 });
const prompt = ChatPromptTemplate.fromTemplate(`
You are the official assistant of Lahore Garrison University (LGU).
Answer ONLY using the context below. If the answer is not in the context,
say: "Sorry, I do not have that information yet." Keep answers short and friendly.

Context:
{context}

Question: {question}
Answer:`);
const chain = prompt.pipe(llm).pipe(new StringOutputParser());

export async function askRag(question) {
  const docs = await retriever.invoke(question);
  const context = docs.map((d) => d.pageContent).join("\n---\n");
  const answer = await chain.invoke({ context, question });
  return { answer, docs };
}
```

Reading tip
temperature: 0 makes the model stable and factual. The prompt line 'Answer ONLY using the context' is what stops hallucination.

7.4 Step 4 - The location router

This is the simple keyword layer. A list of places, each with keywords and coordinates, plus a matcher. It runs **before** RAG, so it is instant and costs nothing.

server/locations.js

```
export const LOCATIONS = [
  { id: "lgu", name: "Lahore Garrison University",
    keywords: ["lgu", "lahore garrison university", "main campus", "university"],
    lat: 31.4640061, lng: 74.4426299 },
  { id: "cafe", name: "LGU Cafe",
    keywords: ["cafe", "cafeteria", "canteen", "food court"],
    lat: 31.4635725, lng: 74.443295 },
  { id: "fountain", name: "LGU Fountain Ground",
    keywords: ["fountain", "fountain ground"],
    lat: 0, lng: 0 },           // TODO: replace with real coordinates (section 8.3)
];

const INTENT = /(where|location|locate|map|navigate|direction|reach|find|way to|how to get)/i;

export function matchLocation(message) {
  const q = message.toLowerCase().trim();
  let best = null, bestLen = 0;
  for (const loc of LOCATIONS) {
    for (const kw of loc.keywords) {
      const hit = new RegExp(`\\b${kw}\\b`, "i").test(q);
      if (hit && kw.length > bestLen) { best = loc; bestLen = kw.length; }
    }
  }
  if (!best) return null;
  const onlyKeyword = q.replace(/[^a-z ]/g, "").trim() === best.keywords.find(
    (k) => k.length === bestLen);                // user typed just "LGU"
  return INTENT.test(q) || onlyKeyword ? best : null;
}
```

Gotcha: why we pick the LONGEST keyword
'where is lgu cafe' contains both 'lgu' and 'cafe'. Picking the longest match (cafe) gives the right answer. Without this rule the bot would always show the university gate.

7.5 Step 5 - The Express server

server/server.js

```
import "dotenv/config";
import express from "express";
import cors from "cors";
import { askRag } from "./rag.js";
import { LOCATIONS, matchLocation } from "./locations.js";

const app = express();
app.use(cors());
app.use(express.json());

app.post("/api/chat", async (req, res) => {
  try {
    const message = (req.body.message || "").trim();
    if (!message) return res.status(400).json({ error: "Message is required" });

    // Layer 1: deterministic location router (instant, free)
    const loc = matchLocation(message);
    if (loc) {
      return res.json({ type: "location", source: "router",
        text: `Here is ${loc.name} on the map.`, location: loc });
    }

    // Layer 2: RAG (semantic search + LLM)
    const { answer, docs } = await askRag(message);

    // Optional: if the best chunk is tagged with a place, attach a map card too
    const tagged = docs.find((d) => d.metadata?.locationId);
    const place = tagged && LOCATIONS.find((l) => l.id === tagged.metadata.locationId);
    res.json({ type: place ? "location" : "text", source: "rag", text: answer,
               location: place || null });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: "Something went wrong" });
  }
});

app.listen(process.env.PORT || 5000, () => console.log("Server running"));
```

Test with Postman: POST http://localhost:5000/api/chat, body `{"message":"where is LGU cafe"}`. You should get **type: location**.

8 Frontend: Chat UI and Map Card

8.1 Create the React app

Terminal (inside lgu-chatbot)

```
npm create vite@latest client -- --template react
cd client
npm install
npm run dev
```

8.2 The map card (the part everyone will notice)

The card embeds a Google Map centered on the coordinates with a **red marker**, and shows two buttons. No API key is required for these URL formats.

client/src/components/LocationCard.jsx

```
export default function LocationCard({ location }) {
  const { name, lat, lng } = location;
  const label = encodeURIComponent(name);
  const embedUrl = `https://maps.google.com/maps?q=${lat},${lng}(${label})&z=18&output=embed`;
  const openUrl = `https://www.google.com/maps/search/?api=1&query=${lat},${lng}`;
  const dirUrl = `https://www.google.com/maps/dir/?api=1&destination=${lat},${lng}&travelmode=walking`;

  return (
    <div style={{ border: "1px solid #cbd5e1", borderRadius: 12, overflow: "hidden",
                  maxWidth: 360, marginTop: 8, background: "#fff" }}>
```

```
      <div style={{ padding: "8px 12px", fontWeight: 700 }}>{name}</div>
      <iframe title={name} src={embedUrl} width="100%" height="200"
              style={{ border: 0 }} loading="lazy" />
      <div style={{ display: "flex", gap: 8, padding: 10 }}>
        <a href={openUrl} target="_blank" rel="noreferrer">Open in Google Maps</a>
        <a href={dirUrl} target="_blank" rel="noreferrer">Get Directions</a>
      </div>
    </div>
  );
}
```

8.3 How to get coordinates for ANY place (kills the 'registered place' blocker)

- Open Google Maps on a computer and zoom to the exact spot (for example the fountain).
- **Right-click** on the spot. The first line of the menu shows numbers like 31.4638, 74.4430.
- Click those numbers. They are copied. First number = lat, second = lng.
- Paste into locations.js. Done. The red pin appears at that exact spot, even if Google does not know a place name there.

8.4 The chat window

client/src/components/ChatBox.jsx

```
import { useState, useRef, useEffect } from "react";
import LocationCard from "./LocationCard";

const API = "http://localhost:5000/api/chat";

export default function ChatBox() {
  const [msgs, setMsgs] = useState([{ role: "bot", text: "Hi! Ask me anything about LGU." }]);
  const [input, setInput] = useState("");
  const [busy, setBusy] = useState(false);
  const endRef = useRef(null);
  useEffect(() => endRef.current?.scrollIntoView({ behavior: "smooth" }), [msgs]);

  async function send() {
    const text = input.trim();
    if (!text || busy) return;
    setMsgs((m) => [...m, { role: "user", text }]);
    setInput(""); setBusy(true);
    try {
      const r = await fetch(API, { method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ message: text }) });
      const data = await r.json();
      setMsgs((m) => [...m, { role: "bot", text: data.text || "Error", location: data.location,
                              source: data.source }]);
    } catch {
      setMsgs((m) => [...m, { role: "bot", text: "Server not reachable." }]);
    }
    setBusy(false);
  }

  return (
    <div style={{ maxWidth: 480, margin: "20px auto", fontFamily: "sans-serif" }}>
      <div style={{ height: 480, overflowY: "auto", padding: 12, background: "#f1f5f9", borderRadius: 12 }}>
        {msgs.map((m, i) => (
          <div key={i} style={{ textAlign: m.role === "user" ? "right" : "left", marginBottom: 10 }}>
            <span style={{ display: "inline-block", padding: "8px 12px", borderRadius: 12,
              background: m.role === "user" ? "#1d4e89" : "#fff",
              color: m.role === "user" ? "#fff" : "#111" }}>{m.text}</span>
            {m.location && <LocationCard location={m.location} />}
            {m.source && <div style={{ fontSize: 10, color: "#64748b" }}>answered by: {m.source}</div>}
          </div>
        ))}
        <div ref={endRef} />
      </div>
      <div style={{ display: "flex", gap: 8, marginTop: 8 }}>
        <input value={input} onChange={(e) => setInput(e.target.value)}
          onKeyDown={(e) => e.key === "Enter" && send()} placeholder="Where is LGU cafe?"
          style={{ flex: 1, padding: 10, borderRadius: 8, border: "1px solid #cbd5e1" }} />
        <button onClick={send} disabled={busy}>{busy ? "..." : "Send"}</button>
      </div>
```

```
    </div>
  );
}
```

client/src/App.jsx

```
import ChatBox from "./components/ChatBox";
export default function App() { return <ChatBox />; }
```

9 Run and Test

- Terminal 1: **cd server, node ingest.js** (first time only), then **npm start**.
- Terminal 2: **cd client, npm run dev**. Open the shown localhost address.
- Try the test cases below. Keep this table, it becomes your evaluation chapter.

#	User types	Expected path	Expected result
1	LGU	Router	Map card for the university, red marker
2	where is LGU cafe	Router	Map card for LGU Cafe (longest keyword wins)
3	where is fountain ground	Router	Map card for Fountain Ground
4	navigate to canteen	Router	Cafe card (synonym keyword)
5	where can I eat lunch?	RAG	Text answer about the cafe, plus map card via metadata tag
6	when does the library open?	RAG	Text answer from knowledge.json
7	who won the cricket world cup?	RAG	'Sorry, I do not have that information yet.' (no hallucination)

10 Presenting and Defending Your FYP

Be honest about the router
The keyword router is not a trick. It is a real, widely used design called **hybrid intent routing**: use cheap deterministic rules for known cases and AI only when needed. Say it openly, explain why you did it (speed, zero cost, 100 percent accurate for known places, no AI mistakes on navigation). That is a strength. Your evaluator will ask how it works, so make sure **you** can explain every box in the diagram.

What to show so the project looks as solid as it is

- **Architecture slide:** use the big-picture diagram from section 3. Name each part: Presentation layer (React), API layer (Express), Intent router, Retrieval layer (embeddings + Atlas Vector Search), Generation layer (LLM).
- **Show the 'answered by' tag** in the UI (router or rag). It proves the system makes a decision on each message.
- **Live demo order:** (1) ask a knowledge question, RAG answers; (2) ask 'where is LGU cafe', the map card appears; (3) click Get Directions, it opens on your phone; (4) ask an unrelated question, the bot refuses politely.
- **Show the database:** open Atlas and show the stored chunks with the embedding arrays. Explain 1536 dimensions in one sentence.
- **Show the test table** from section 9 with pass or fail results, as your evaluation.

- **Add one small real feature:** the Get Directions button. It uses the visitor's own location with one line of code and always impresses.
- **Add an admin file or page** for adding new places (locations.js now, a MongoDB collection later). It shows the system is extendable.

Likely viva questions and honest answers

Question	Short answer
Why RAG and not fine-tuning?	Fine-tuning is costly and bakes data into the model. RAG lets us update knowledge by editing a file and re-running ingest, and reduces hallucination.
How does the bot know the answer?	The question is converted to an embedding, the closest chunks are found in Atlas, and the LLM answers only from those chunks.
Why a keyword router in front?	Navigation needs exact coordinates. Rules are instant, free and never wrong. The LLM could guess a wrong place.
What if the user types a place not in your list?	It falls to RAG. Future work: use Google Places or Geocoding API for campus-wide search.
How do you stop hallucination?	Strict prompt, temperature 0, and only retrieved context is given. Test case 7 proves it.
Is your data safe?	API key is in .env on the server only. The frontend never sees it.
What are the limitations?	Only known places have map cards, English only, depends on OpenAI uptime and credits.
Future work?	Urdu support, voice input, live bus tracking, indoor maps, admin dashboard.

11 Troubleshooting and Final Checklist

Problem	Likely cause and fix
401 Incorrect API key	Key copied wrongly or .env not loaded. Make sure import "dotenv/config" is the first line and the file is in the server folder.
429 You exceeded your quota	No credits left. Add credits in Billing (section 4.2).
Vector search returns empty	Index not Active yet, wrong index name, or numDimensions is not 1536. Check that the path is 'embedding'.
MongoServerSelectionError	IP not whitelisted in Network Access, or the password in the URI has special characters (URL-encode them).
CORS error in browser	Missing app.use(cors()) or wrong port in the frontend API constant.
Map shows grey or blank	Coordinates swapped (lat and lng reversed) or still 0,0. Latitude about 31.4, longitude about 74.4 for Lahore.
Always shows the university card	Router matched 'lgu' every time. Keep the longest-keyword rule in matchLocation.
Cannot use import statements	Add "type": "module" to server/package.json.

Final checklist

- OpenAI account, credits loaded, usage limit set, key saved in .env

- Atlas cluster, vector_index Active (1536, cosine)
- knowledge.json filled with real LGU facts, ingest.js run successfully
- locations.js has real coordinates for every demo place (right-click method)
- Postman test passes, React demo passes all 7 test cases
- Architecture diagram, test table and viva answers ready
- .gitignore protects .env and you can explain every file

You are done
Remember the one sentence that explains the whole project: *'A hybrid chatbot that uses rules for exact campus navigation and RAG for open-ended university questions.'*